Solution: Exercises for the Lecture Advanced Methods of AI

Asking, Defining, Constraining, Deriving, Generating

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Last week you *represented* the pizza world in first-order logic and in Python. Tonight a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression, a **description**, written in the Entity Query Language (EQL). The five acts of the evening are five things the robot does with one:

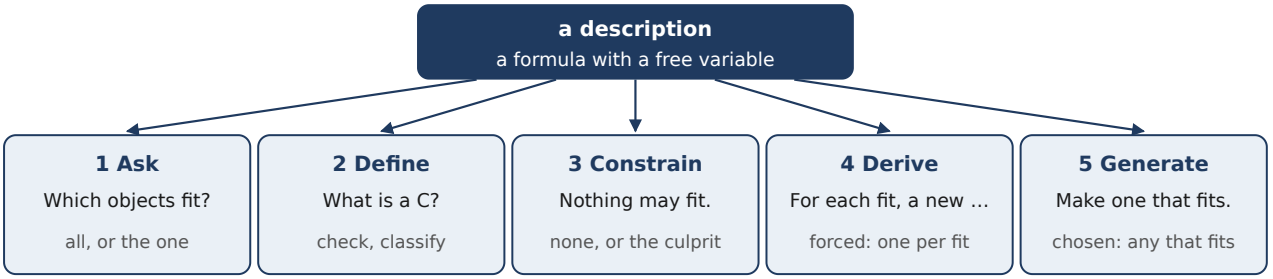


Figure 1. One description, five uses. The expression stays the same; what changes is what is done with it. Asking and constraining *read* what the robot knows; defining, deriving and generating *write* to it.

How to work

The evening happens in the notebook `03-pizza_queries.ipynb` (link on Stud.IP). Open it in pairs, run its setup cell, and work through it from top to bottom.

Three times, the tutor calls a **pen break**. Put the laptop aside and turn to this sheet: write the logic by hand for five minutes. Then the tutor writes an answer on the board and takes your questions, and you go back to the notebook. Questions marked ★ are for pairs who finish early.

FOR THE TUTOR

The 90 minutes

Time	Part	Students	Tutor, and the sentence that leads on
0–8	Slides	open the notebook and start its setup cell, then listen	slides 1–9. *“Let us look at the world the robot knows.”*
8–30	Prologue, Act 1 Asking	notebook: the world, 1(a)–(e), with their predictions	walk around; at 1(d) ask the room: what should the robot do with 0, and with 2? *“Pens out: pen break 1.”*
30–37	**Pen break 1**	the sheet, page 2: write for 5 minutes	board: (b), Q1 and plain_1; then questions, and the surprises listed below the pen break. *“You asked which pizzas are vegetarian. What does the word mean?”*
37–49	Act 2 Defining	notebook: 2(a)–(c)	at 2(c), ask about napoletana_1: is “no label” the same as “not vegetarian”? *“Some descriptions must never fit anything.”*
49–57	Act 3 Constraining	notebook: 3(a)–(c)	one pair shows its red line in the picture. *“Pens out: pen break 2.”*
57–64	**Pen break 2**	the sheet: write for 5 minutes	board: (c), the constraint and $\neg\forall \equiv \exists\neg$; draw $\leftrightarrow$ with $\leftarrow$ classify, $\rightarrow$ conclude. *“The labels were facts about pizzas. Now the kitchen writes new things.”*
64–73	Act 4 Deriving	notebook: 4(a), 4(b)	4(b) is the climax: three pairs read out one output each. *“So far, every new object was forced. Dana orders one she chooses.”*
73–79	Act 5 Generating	notebook: Act 5	*“Pens out, one last time.”*
79–90	**Pen break 3**	the sheet: (a) and (b), then the summary together	collect (b) on the board: several answers are right; fill the summary with the room; show Figure 11. Then the notebook's bill.

If time runs short

Skip the ★ questions and assign the summary of pen break 3 for home. Pairs that are fast in the notebook go on; nobody waits. Every notebook cell runs as it is, so a pair that is stuck on a `TODO` goes on and compares with the solution later.

PEN BREAK 1 · AFTER ACT 1

Asking

5 minutes of writing, then the board.

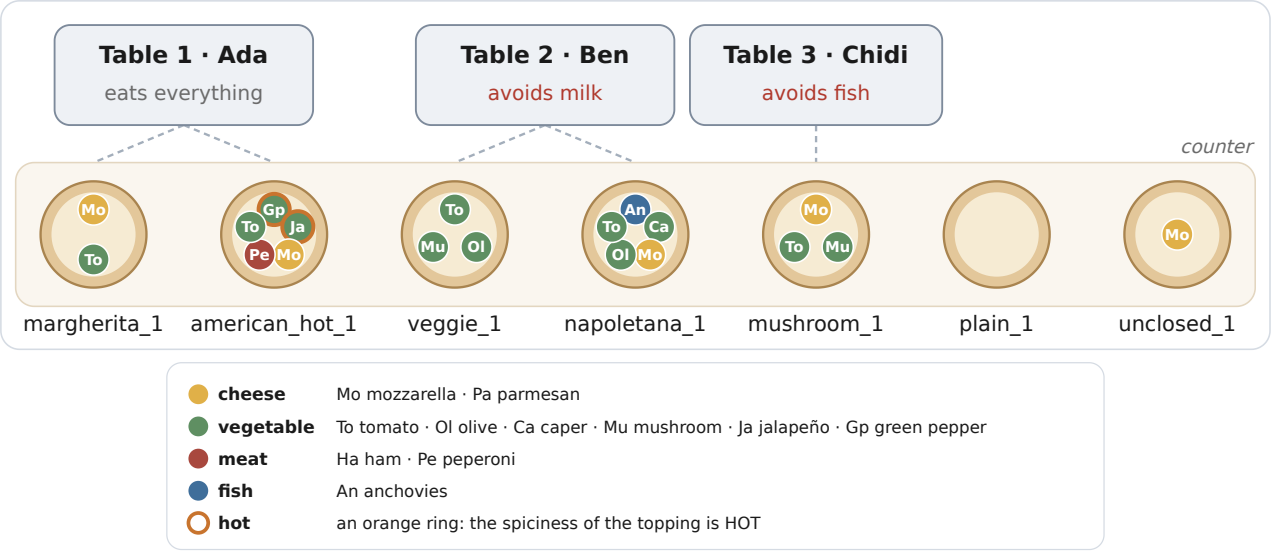


Figure 2. What the robot knows at 19:30. Lines connect each table to the pizzas it ordered.

(a) The picture is an interpretation

Figure 2 is an *interpretation* of first-order logic. What is its domain? What is the extension of the predicate CheeseTopping, and that of the relation hasTopping?

SOLUTION

The domain is the set of all objects: 7 pizzas, their 19 toppings, 3 guests and 3 orders (the spiciness values and allergens are named by constants). The extension of CheeseTopping is the set of the 5 yellow dots, one mozzarella on each cheesy pizza (Figure 3); that of hasTopping is the set of pairs (pizza, dot on that pizza), 19 pairs. Every query tonight is evaluated in this *one* interpretation: a query checks a model, it does not ask whether a formula follows from a theory.

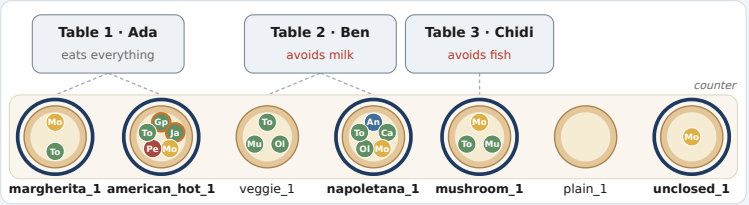


Figure 3. The cheesy pizzas of Act 1(a), ringed.

(b) Some, only, none

```
Q1 = a(Pizza)(toppings=every_is(VegetableTopping)).from_(pizzas)
Q2 = a(Pizza)(toppings=[a(MozzarellaTopping), a(TomatoTopping)]).from_(pizzas)
```

Write Q1 as a formula with the free variable x. The robot returned plain_1, a pizza without any topping: show with your formula why plain_1 satisfies it. Then write Q2 as a formula.

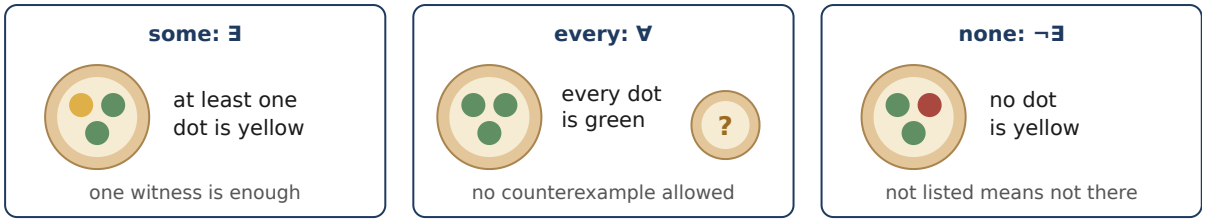


Figure 4. The three quantifier patterns, as the robot checks them on one pizza. The small pizza has no toppings at all: is every dot on it green?

SOLUTION

Q1: Pizza(x) ∧ ∀t (hasTopping(x, t) → VegetableTopping(t)). plain_1 has no toppings, so hasTopping(plain_1, t) is false for every t, and the implication is true for every t: there is no counterexample (vacuous truth). It is p3 of Worksheet 2.

Q2: Pizza(x) ∧ ∃t_1 ∃t_2 (toppings(x) = [t_1, t_2] ∧ MozzarellaTopping(t_1) ∧ TomatoTopping(t_2)). A list of patterns describes the whole list. This implies the closure of Worksheet 2 (some mozzarella, some tomato, and nothing else) and is stronger: it also fixes the number and the order of the toppings, so only margherita_1 fits.

(c) Following the relations

Write `hot_orders`, the orders that contain a pizza with a hot topping, as a formula with the free variable o. The query `set_of(order.guest, pizza, topping)` of the notebook had two answers where `hot_orders` had one: which variables are free in each?

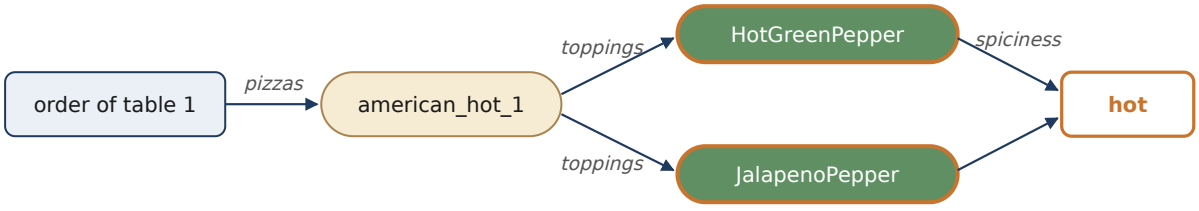


Figure 5. The paths from Ada's order to a hot topping. There are two.

SOLUTION

Order(o) ∧ ∃x (hasPizza(o, x) ∧ ∃t (hasTopping(x, t) ∧ spiciness(t) = hot)). Only o is free; x and t are bound by ∃, so the order is returned once, however many witnesses there are. In the `set_of` query, the guest, x and t are free: an answer is an **assignment** of values to the free variables, and there is one per path of Figure 5.

(d) Making a description precise

“The pizza with olives” fitted two pizzas; Ben wants the one *without fish*. Write the precise description as a formula.

SOLUTION

Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ OliveTopping(t)) ∧ ¬∃t' (hasTopping(x, t') ∧ FishTopping(t')); tonight only veggie_1 satisfies it. Two quantifiers, two variables. Without the ¬∃, as in `pizza.where(not_(HasType(topping, FishTopping)))`, the unquantified variable means *some*: a pizza with at least one topping that is not fish, which napoletana_1 has too.

★ THINK

Last week the OWL reasoner refused to classify UnclosedPizza (a pizza with some mozzarella) as vegetarian. Tonight the query `vegetarian` returned unclosed_1. Who is right, the query or the reasoner?

SOLUTION

Both; they answer different questions. The query evaluates the formula in the one interpretation the robot has, where the list of toppings is complete (closed world). The reasoner asks whether "vegetarian" *follows* from what was said, i.e. holds in every model of it; there are models in which the pizza also has ham (open world). A robot looking at a pizza usually wants the closed-world answer, provided its knowledge of the pizza is complete.

ALSO AT THE BOARD: THE SURPRISES OF ACT 1

- **Q3**, no cheese, returned veggie_1 and plain_1: nobody stated "veggie_1 has no cheese"; the stored list is taken to be complete (closed-world assumption).
- **The path with `not_`** returned every pizza: `not_` negates the condition on *one* topping, ∃t (… ∧ spiciness(t) ≠ hot); *no* hot topping is ¬∃, `none_is`.
- ***a* or *the*:** ham fits nothing (NoSolutionFound), fish one pizza, olives two (MultipleSolutionFound). Russell analysed "the F is G" as ∃x (F(x) ∧ ∀y (F(y) → $y = x$) ∧ G(x)) and calls it false when there is no F or more than one; Strawson says its presupposition fails, so it is neither true nor false. The robot, which refuses instead of answering, takes Strawson's side. Whether a description refers also depends on the domain: "the pizza with cheese of table 3" does.

PEN BREAK 2 · AFTER ACT 3

Defining and constraining

5 minutes of writing, then the board.

(a) A class declaration says part of what a thing is

Cell 2(a) of the notebook printed what KRROOD reads from the class declarations. Write each line as a formula. Can any of them ever make an object a Pizza, or a VegetarianPizza?

Read from the declarations	First-order formula
every MozzarellaTopping is a CheeseTopping	$\forall x$ (MozzarellaTopping(x) → CheeseTopping(x))
every Pizza has toppings, each a PizzaTopping	
every Order has a guest, which is a Guest	
every MenuLabel is a role of one Pizza	

Hint: "has *r*, each a B" is a *relation* $r(x, y)$ between an A and each of its Bs; "has a *r*, which is a B" is a *function* $r(x)$ whose value is a B; "is a role of one B" is a function from the role to the B that takes it. For the question, look at what stands on the right of →.

SOLUTION

Read from the declarations	First-order formula
every Pizza has toppings, each a PizzaTopping	$\forall x\ \forall t$ (Pizza(x) ∧ hasTopping(x, t) → PizzaTopping(t))
every Order has a guest, which is a Guest	$\forall o$ (Order(o) → Guest(guest(o))); guest is a function
every MenuLabel is a role of one Pizza	$\forall l$ (MenuLabel(l) → Pizza(roleTaker(l)))

All four are **necessary** conditions: they say what follows *from* being a C, never when something *is* a C. None of them can make an object a pizza. In Python the class is asserted when an object is created, and `HasType` is `isinstance`: it reads the asserted class and never infers one. These are the subclass and range axioms of pizza.owl from last week.

(b) Concluding from a name

The menu's definition from the notebook: $\forall x$ (VegetarianPizza(x) ↔ Pizza(x) ∧ ¬∃t (hasTopping(x, t) ∧ (MeatTopping(t) ∨ FishTopping(t)))). A pizza from the other branch arrives with a green V; nobody wrote down its toppings. What may the robot tell Chidi, who avoids fish? And Ben, who avoids milk?

Which direction of ↔ do you use?

SOLUTION

Chidi: it is safe, by the necessary direction (→): VegetarianPizza(x) → ¬∃t (hasTopping(x, t) ∧ FishTopping(t)) follows from the name alone. Ben: nothing, because the definition says nothing about cheese. No query of Act 1 could answer this, since the toppings are unknown. A definition works in both directions: ← to classify (the green V of Act 2), → to conclude from the class.

(c) A constraint is a query that must have no answer

The kitchen's rule: every pizza of an order is safe for the guest of that order. Write it as a formula (with hasPizza, guest and IsSafeFor), and then its negation, with the ¬ pushed inwards. Which of the two did `dangerous` ask?

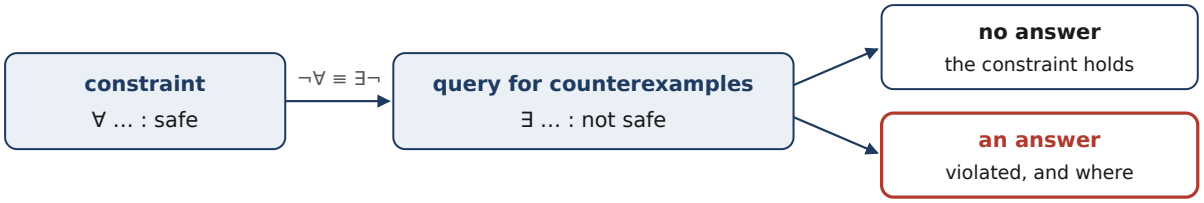


Figure 6. A universal statement is violated exactly when a counterexample exists.

SOLUTION

Constraint: ∀o ∀x (hasPizza(o, x) → IsSafeFor(x, guest(o))).
Negation: ∃o ∃x (hasPizza(o, x) ∧ ¬IsSafeFor(x, guest(o))). `dangerous` asks the negation, with o and x left free, so that each answer names a culprit: table 2, Ben, napoletana_1. EQL could evaluate the ∀ directly, but the answer would only be “false”; the counterexample query says which plate not to serve.

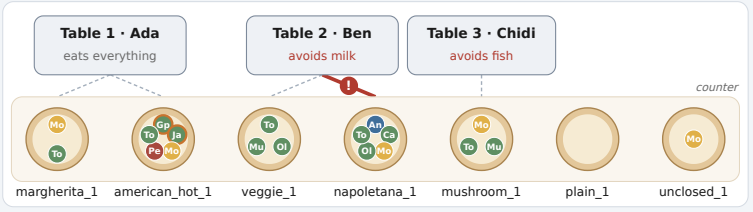


Figure 7. The one violation of the constraint.

★ THINK

Suppose the kitchen's rule were an axiom of last week's ontology. What would the reasoner report for tonight's facts? And if the mozzarella of the Napoletana had simply not been recorded?

SOLUTION

An axiom does not check facts; it restricts the models. Facts that contradict it leave no model at all: the ontology is **inconsistent**, and from an inconsistent theory every formula follows. The reasoner reports the inconsistency, not a plate. If the mozzarella had not been recorded, there would be no contradiction: in the open world, the reasoner would rather conclude that the Napoletana is safe for Ben, because the axiom demands it. Used as an **integrity constraint**, the same formula is checked against the one interpretation; the knowledge base stays usable, and the answer is the culprit.

ALSO AT THE BOARD: THE SURPRISES OF ACTS 2 AND 3

- **Checking and classifying:** frozen_2 is accepted, frozen_1 not. A check is the formula with its variable bound, $\varphi(a)$: yes or no. Classifying applies $\leftarrow$ as a rule, $\forall x\,(\varphi(x) \rightarrow \text{VegetarianPizza}(x))$. The label is a role: an object of its own that stands for the pizza, so the rule adds a fact about something that exists, not a new thing in the kitchen.
- **Registry or domain:** the registry also answers frozen_2, which is not on tonight's menu. The registry is everything the robot has created; `.from_(…)` is the domain the question is about.
- **Rules with exceptions:** napoletana_1 gets no label (Figure 8). The rules are *silent*, and silence is not "no": reading it as "not vegetarian" would be one more closed-world step. The *unless* branch is an exception that replaces the general conclusion; such rule trees are known as ripple-down rules.
- **Stored or computed:** hasTopping and guest are stored, the class hierarchy is declared, the allergens and IsSafeFor are computed. In a query, both look the same; giving a symbol a program as its meaning is called **procedural attachment**.

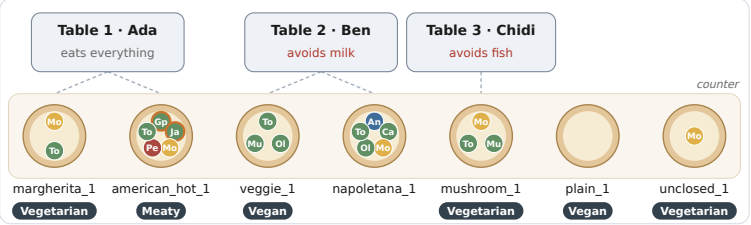


Figure 8. The menu labels, as derived by the rule tree.

PEN BREAK 3 · AFTER ACT 5

Deriving, generating, and the whole evening

5 minutes of writing for (a) and (b); the summary together with the tutor.

(a) A rule with ∃ in its head

The milk rule of Act 4 is

```
∀x (Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t)) → ∃w (Warning(w) ∧ about(w,
x) ∧ allergen(w) = milk))
```

Write the fish rule the same way. The label rule of Act 2, $\forall x\,(\varphi(x) \rightarrow \text{VegetarianPizza}(x))$, only labelled pizzas that were there; the milk rule created five new warnings. What in the formula makes the difference?

SOLUTION

$\forall x\,(\text{Pizza}(x) \wedge \exists t\,(\text{hasTopping}(x, t) \wedge \text{FishTopping}(t)) \rightarrow \exists w\,(\text{Warning}(w) \wedge \text{about}(w, x) \wedge \text{allergen}(w) = \text{fish}))$. The left side of $\rightarrow$ is the **body**: a description, as in Act 1. The right side is the **head**: for every answer of the body, a new object w is created as the witness of $\exists w$. The label rule's head has no $\exists$: it only adds facts about the objects there are.

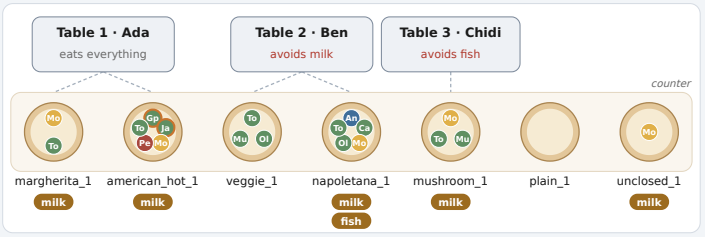


Figure 9. The milk and fish warnings (gluten, on every pizza, is left out).

(b) One plan, many answerers

The robot's next task is a plan with open slots, each written as a description:

Bring ① a pizza that is safe for Ben, lying ② at some pose on the counter, **to** ③ the table of Ben, **holding the plate with** ④ a grasp that works. **If none is on the counter:** ⑤ have one made.

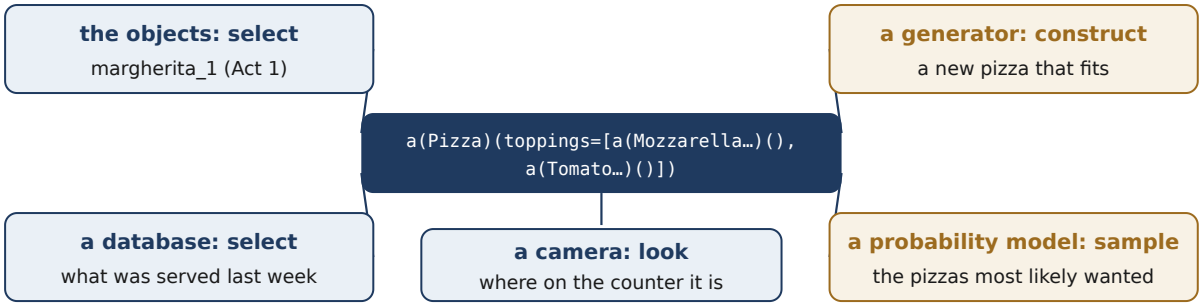


Figure 10. The description is written once, symbolically. Selecting (blue) and constructing or sampling (ochre) are different ways of answering it.

Which kind of backend (Figure 10) should answer each slot? Choose from **K** a query over the knowledge base, **R** rules and predicates, **L** looking with the camera, **S** sampling from a probability model, **G** a generator.

SOLUTION

① safe pizza	② its pose	③ Ben's table	④ grasp	⑤ have one made
R and K: IsSafeFor over the pizzas	L: the knowledge base knows *that* it is there, not exactly *where*	K: the orders	S: sampled, then tried	G: Act 5

Several answers are defensible; what matters is that the plan does not say *who* answers. It states what it needs, symbolically, and each slot goes to the backend that can answer that kind of description. The robots of our current research work this way: in a sorting task, the piece to pick up is *looked for* by perception, where it stands is *read off* the robot's digital twin, the hole it fits through is *concluded* by rules, and how to grasp it is *sampled*: one plan, four backends.

(c) The whole evening

For each use of a description, give tonight's example and what it is in logic.

SOLUTION

Use	The question behind it	Tonight's example	In logic
1 Ask	Which objects fit?	the cheesy pizzas; who gets which hot topping; *the* pizza with olives and without fish	the assignments to the free variables that satisfy a formula in one interpretation; *the*: exactly one
2 Define	What is a C?	the green V: frozen_2 accepted, five pizzas classified; the menu labels	C(x) ↔ φ(x): ← classifies (a rule without ∃ in its head), → concludes from the class; class declarations give necessary conditions only
3 Constrain	Does anything violate it?	no pizza unsafe for its guest	a ∀ checked through its counterexamples ∃¬; an integrity constraint, not an axiom
4 Derive	What new object does each fit get?	the allergen warnings	a rule with ∃ in its head: one new witness per answer of the body, by forward chaining
5 Generate	What could fit?	Dana's pizza; the slots of the plan	constructing new objects that make the description true, chosen among many

Two questions tell the uses apart. Does the use **read** the knowledge base (1, 3) or **write** to it (2, 4, 5)? And must the answer fit the world, as for a question, or must the world fit the formula, as for a constraint and a specification? Deriving and generating both create objects: a rule's objects are *forced*, one per answer of its body; a generator *chooses* one of many that fit, so it has to search.

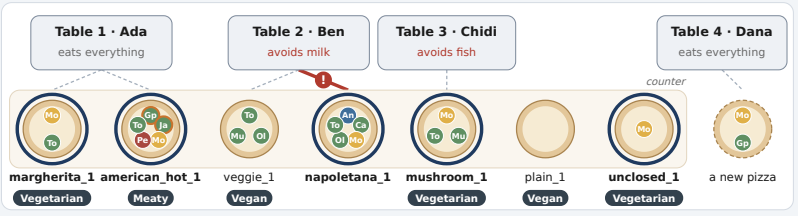


Figure 11. What the robot knows at 20:00: the answers to the chef's question (ringed), the menu labels, the violated constraint, and Dana's new pizza.

ALSO AT THE BOARD: THE SURPRISES OF ACTS 4 AND 5

- **The world moves on:** after the parmesan, only the constraint protects Ben; it is evaluated against the world as it is *now*. The stored milk warnings are stale until the rule runs again: keeping derived facts in line with their reasons is **truth maintenance**. “Vegan” was withdrawn because a fact was *added*: the reasoning is **non-monotonic**, because the vegan rule relies on the absence of cheese; classical first-order logic is monotonic.

- **Forward and backward:** the warnings were derived in advance (**forward chaining**, materialisation); the predicate of Act 3 derives its conclusion only when a question needs it (**backward chaining**).
- **Generating:** an answer is now a new object that, added to the interpretation, makes the description true: answering means *constructing* a model instead of checking one. Dana's open description has three answers, one per spiciness. Asked again, `hot_pizzas` also finds the kitchen's hot offer: every evaluation reads the registry as it is then, while questions with `.from_(pizzas)` keep their answer.

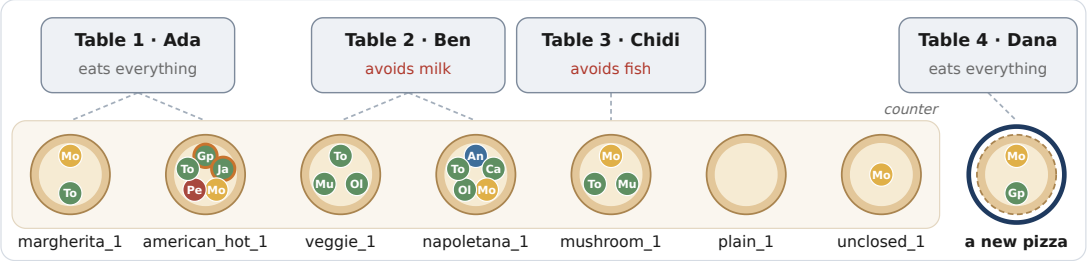


Figure 12. The world has grown: pizzas that did not exist at 19:30.

Take home

One expression, a description, carried the whole evening. Its meaning is a formula with free variables; what you *do* with it decides whether it is a question, a definition, a constraint, the body of a rule or a specification, and which backend answers it decides whether it is checked, computed or constructed.